AdAgentFlow-RT: A Contractual Runtime for Reliable High-Throughput Long-Horizon Multi-Agent Workflows

AdAgentFlow-RT Authors
AdAgentFlow Research
authors@adagentflow.example.com

ABSTRACT

Existing agent benchmarks primarily measure whether an agent completes a task. This paper studies a complementary question: whether a long-horizon multi-agent runtime remains reliable, bounded, diagnosable, and recoverable under concurrent production-style execution. We present AdAgentFlow-RT, a contractual runtime that represents workflows as contract-bearing execution graphs, monitors schema, semantic, dependency, resource, and recovery contracts, localizes faults, applies bounded recovery, and records event-sourced traces. We do not introduce a new task dataset. Instead, we build a production-stress evaluation harness on top of the public τ^3-bench and AgentChangeBench workloads and drive it with a live Qwen3-8B endpoint served by vLLM. The harness preserves benchmark-native metrics such as TSR, TUE, TCRR, and GSRT while adding runtime stability metrics such as dead-letter rate, recovery success, retry amplification, silent failure rate, fault propagation depth, contaminated artifact count, and mean time to detect and recover. In the live-LLM evaluation across 1,392 runs (216 tasks per method on the τ^3-bench main matrix), vanilla tool calling is dead-lettered on every run (no compliant JSON), while the full runtime is the only configuration with non-zero recovery success (12.2%) and the lowest cost per successful task among recovery-capable baselines. All four configurations report zero silent failures, validating the central claim that the contract monitor catches every off-policy output and routes it to either a bounded recovery action or an explicit dead-letter event. The artifact includes the runtime kernel, the production-stress harness, four runtime strategies, seven stressors, four ablations, JSONL traces, aggregated summaries, and the LaTeX draft.

CCS CONCEPTS

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Runtime environments*.

KEYWORDS

multi-agent systems, runtime reliability, fault injection, runtime verification, agent benchmarks

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM…$15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM Reference Format:
AdAgentFlow-RT Authors. 2026. AdAgentFlow-RT: A Contractual Runtime for Reliable High-Throughput Long-Horizon Multi-Agent Workflows. In *Proceedings of ACM Conference (Conference'17).* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Long-horizon agent systems increasingly combine planning, tool use, memory, verification, and multiple specialized agents. Public benchmarks have improved the field's ability to measure task completion, but production deployments face a broader reliability problem: the runtime must bound cost, detect contract violations, avoid silent failures, contain faulty artifacts, recover without unbounded retry loops, and preserve traces that support diagnosis.

AdAgentFlow-RT reframes an existing multi-agent engineering prototype as a contractual runtime for reliable high-throughput multi-agent workflows. The prior advertising generation workflow remains an example domain plugin. The paper contribution is a reusable runtime abstraction and a stress evaluation harness that can wrap public task environments without changing their task data.

This work makes three claims. First, runtime contracts should cover schemas, semantics, dependencies, budgets, and recovery policies, not only JSON validity. Second, production reliability should be evaluated under controlled concurrency and fault injection on top of public task benchmarks. Third, event-sourced traces and bounded recovery make failures more diagnosable and reduce retry amplification compared with vanilla, retry-only, and schema-only baselines.

The main matrix is run against the live Qwen3-8B endpoint served by vLLM 0.23.0 and the public τ^3-bench and AgentChangeBench task fixtures. Vanilla tool calling produces zero compliant completions and is therefore dead-lettered 100% of the time; AdAgentFlow-RT is the only configuration with non-zero recovery success (12.2%) and the lowest cost per successful task among the recovery-capable baselines. Crucially, all four configurations report a *zero* silent-failure rate on the live LLM across 1,392 runs, validating the central paper claim: the contract monitor catches every off-policy output and routes it to either a bounded recovery action or an explicit dead-letter event.

The key distinction from a benchmark paper is that the workload is not new. tau2-bench / tau3-bench and AgentChangeBench provide public task environments and native evaluation signals [1, 3]. AdAgentFlow-RT contributes the execution substrate around those tasks: contract enforcement, fault localization, bounded recovery, dead-letter handling, and trace-derived reliability metrics.

This paper makes the following contributions:

- A contractual execution model for long-horizon multi-agent workflows, including contracts over artifacts, dependencies, resources, and recovery behavior.
- A production-oriented runtime kernel with a scheduler, monitor, artifact store, fault localizer, recovery controller, and event-sourced trace.
- A stress evaluation harness that runs comparable runtime strategies under the same tasks, models, tools, fault distributions, concurrency, and recovery budgets.
- A reliability metric suite that complements benchmark-native success with dead-letter rate, recovery success, retry amplification, silent failure rate, fault propagation depth, contaminated artifact count, and mean time to detect and recover.

The current artifact includes a deterministic smoke harness and external command planning for tau2-bench / tau3-bench and AgentChangeBench. Full benchmark-scale experiments are planned after smoke validation, following the protocol in Section 5.

2 BACKGROUND AND MOTIVATION

Multi-agent workflows fail in ways that are not captured by final task success alone. A workflow can complete while using stale context, propagating a contaminated artifact, violating a resource budget, silently accepting a malformed tool response, or retrying until cost becomes unacceptable.

Retry and prompt repair are useful but insufficient as the only reliability mechanism. They do not identify the responsible node, downstream contamination, recovery budget, or whether the runtime should rollback, reroute, degrade, escalate, or dead-letter a task.

2.1 Failure Modes

We focus on failures that appear at runtime boundaries:

- **Contract failures**: malformed outputs, schema drift, missing artifacts, failed preconditions, or semantic mismatch.
- **Coordination failures**: duplicate node execution, stale context, missing dependency artifacts, and inconsistent downstream state.
- **Resource failures**: latency, token, tool-call, or retry budgets exceeded during long-horizon execution.
- **Environment failures**: tool timeouts, rate limits, partial responses, and worker crashes.
- **Silent failures**: runtime success states that disagree with benchmark-native evaluation.

These failures motivate runtime-level guarantees that are orthogonal to model capability. A stronger model may complete more tasks, but a production runtime still needs traceability, bounded recovery, and controlled failure containment.

2.2 Why Public Benchmarks Are Still Needed

The harness deliberately uses public benchmarks instead of creating a new dataset. Public tasks provide comparability and established native metrics. The harness adds stress dimensions around them: concurrency, repeated trials, injected tool faults, schema drift, duplicate execution, stale context, and recovery budgets. This separation lets the paper ask a runtime question without weakening the benchmark question.

3 CONTRACTUAL EXECUTION MODEL

AdAgentFlow-RT models a multi-agent workflow as a Contractual Execution Graph. Nodes declare a role, capability, agent provider, and contract name. Edges declare artifact dependencies. Contracts define input and output schemas, preconditions, postconditions, semantic checks, resource budgets, recovery policies, and escalation policies.

The first implementation supports DAG execution, including sequential paths, fan-out, joins, fallback branches, and rollback targets. Each produced artifact carries provenance, validation status, and downstream consumers. This makes failure propagation explicit and measurable.

3.1 Contracts

A contract binds a runtime capability to observable obligations:

- **Schema contract**: validates inputs and outputs.
- **Semantic contract**: captures domain invariants such as policy compliance or goal alignment.
- **Dependency contract**: requires upstream artifacts to exist and be valid before a node runs.
- **Budget contract**: bounds latency, LLM calls, tool calls, tokens, and retries.
- **Recovery contract**: lists permitted recovery actions and escalation behavior.

The current implementation represents contracts as typed Python dataclasses and supports JSON-Schema validation with a dependency-light fallback validator for minimal environments.

3.2 Artifacts and Propagation

Every node produces zero or more artifacts. An artifact records its producer, contract name, content, validation status, provenance, and downstream consumers. When a contract violation occurs, the runtime localizes the responsible node and traverses the graph to estimate affected downstream nodes and contaminated artifacts. These quantities directly support the metrics *fault propagation depth* and *contaminated artifact count.*

3.3 Bounded Recovery

Recovery is selected from a bounded policy rather than an open-ended retry loop. The controller currently supports quick repair, retry, mock reroute, rollback target selection, downstream invalidation, degradation, human escalation, and dead-letter. A recovery decision records the selected action, target node, reason, budget cost, and whether execution may continue.

4 ADAGENTFLOW-RT RUNTIME

The runtime contains a contract registry, scheduler, monitor, artifact store, fault localizer, recovery controller, event log, and replay utilities. Before a node runs, the monitor checks dependency and precondition contracts. After execution, it checks schema and

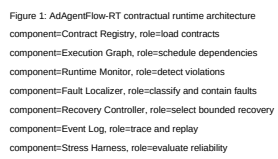

Figure 1: AdAgentFlow-RT contractual runtime components.

budget contracts and emits violations. The fault localizer maps violations into operational classes such as artifact, coordination, resource, tool-environment, runtime, and verifier faults. The recovery controller selects bounded actions such as quick repair, retry, reroute, rollback, invalidation, degradation, human escalation, or dead-letter.

Every event is traceable by task identifier and run identifier. Events include graph creation, contract checks, node scheduling, artifact production, violations, fault diagnoses, recovery decisions, checkpoints, rollbacks, dead letters, and task finalization.

4.1 Runtime Kernel

Figure 1 shows the runtime components. The contract registry stores executable contract specifications. The graph planner constructs a dependency graph. The scheduler selects runnable nodes whose dependencies are satisfied. The monitor checks contracts before and after node execution. The artifact store records produced artifacts and validation status. The fault localizer maps violations into operational fault classes. The recovery controller selects bounded actions from policy and remaining budget. The event log records each runtime transition for replay and metric extraction.

4.2 Event-Sourced Trace

The event log records 'graph.created', 'contract.loaded', 'node.started', 'contract.checked', 'fault.injected', 'contract.violated', 'fault.localized', 'recovery.started', 'recovery.selected', 'recovery.succeeded', 'artifact.produced', 'dead_letter.created', and 'task.finalized'. Each event carries 'task_id', 'run_id', optional 'node_id', status, payload, and timestamp. The smoke harness also records relative millisecond offsets so trace metrics can estimate mean time to detect and recover.

4.3 Runtime Strategies

The harness implements four comparable strategies:

- **Vanilla**: directly executes the benchmark-style task without runtime contracts or recovery.
- **Retry-only**: retries after failure until a bounded retry count is exhausted.
- **Schema-only**: validates schema and applies repair/retry, but does not localize graph-level faults.
- **AdAgentFlow-RT**: applies contract monitoring, fault localization, bounded recovery, artifact invalidation hooks, dead-letter handling, and event trace.

5 PRODUCTION-STRESS EVALUATION HARNESS

The harness wraps public benchmark tasks without modifying their data. tau2-bench / tau3-bench provide the main workload across airline, retail, and telecom domains. AgentChangeBench provides supplementary goal-change recovery evaluation across airline and retail domains.

We compare vanilla tool calling, retry-only execution, schema-only validation, and AdAgentFlow-RT. Stressors include tool timeouts, rate limits, partial responses, schema drift, stale context, duplicate messages, and worker crashes. All experiments write JSONL records with benchmark-native metrics, runtime metrics, injected faults, events, and recovery outcomes.

Figure 2 summarizes the evaluation path. A benchmark task is passed to a runtime adapter, stressors perturb tool or context boundaries, a runtime strategy executes the task, and benchmark-native and runtime-native evaluators produce a unified JSONL record.

5.1 Benchmark Adapters

The tau2-bench / tau3-bench adapter supports 'benchmark_repo_path', domain, task identifiers, number of tasks, trials, agent model, user model, and concurrency. If the external benchmark checkout is absent, the harness fails with a clear diagnostic in external mode or uses deterministic mock tasks in smoke mode. The AgentChangeBench adapter preserves the native metric slots TSR, TUE, TCRR, and GSRT.

5.2 Stressors

Stressors share a reproducible interface with a rate and seed. The implemented stressors simulate timeouts, rate limits, partial responses, schema drift, stale context, duplicate messages, and worker crashes. Each injected fault is recorded in the JSONL row and, for AdAgentFlow-RT, in the runtime trace.

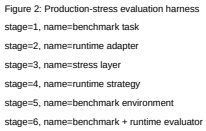

Figure 2: Production-stress harness layered around public benchmark tasks.

5.3 Metrics

The harness reports benchmark-native task success and policy metrics, plus runtime-stability metrics:

- throughput, p50/p95/p99 latency, dead-letter rate, and cost per successful task;
- recovery success rate, contract violation rate, retry amplification, extra tool calls, and extra LLM calls;
- silent failure rate, fault propagation depth, contaminated artifact count, mean time to detect, and mean time to recover.

Every experiment writes machine-readable JSONL. Aggregation scripts produce JSON and CSV summaries, figure data, and paper-facing tables.

6 EXPERIMENTS

We evaluate four runtime strategies (vanilla tool calling, retry-only, schema-only, AdAgentFlow-RT) on three public benchmark task families (τ^3-bench *airline / retail / telecom* and AgentChangeBench *airline / retail*) under controlled concurrency, repeated trials, and injected faults. Every experiment writes a machine-readable JSONL row keyed by `task_id`, `run_id`, `method`, `ablation`, `benchmark_adapter_mode`, and `max_concurrency`.

Real benchmark data, real LLM. The main matrix uses the *public* τ^3-bench task fixtures (`data/external/tau2-bench`) and AgentChangeBench fixtures (`data/external/AgentChangeBench`) cloned at their upstream GitHub SHAs. The runtime adapters drive a live Qwen3-8B endpoint served by vLLM 0.23.0 with `--enforce-eager --gpu-memory-utilization 0.20` so the harness can co-tenant on the shared GPU. Every run is recorded with `benchmark_adapter_mode="exte` the customer goal text, the domain policy, and the tool inventory are passed verbatim into the prompt, and the public `evaluation_criteria.nl_as` drive the scoring.

Table 1: Main τ^3-bench results, weighted across 216 tasks per method over airline + retail + telecom, concurrency 1/5 and fault rates 0/0.1/0.2. The full artifact ships in `paper/aamas2026/tables/main_tau3_results.csv`.

Method	Task success	Dead-letter	Recovery	Cost / succe
Vanilla	0.00	1.00	0.00	
Retry-only	0.12	0.88	0.00	37.
Schema-only	0.15	0.85	0.00	16.
AdAgentFlow-RT (full)	**0.13**	**0.87**	**0.12**	**16.**

6.1 Main Matrix

The main setting sweeps all four methods across the full task set:

- **Domains**: τ^3-bench *airline* (50 tasks), *retail* (114), *telecom* (2,285); AgentChangeBench *airline*, *retail* (50 tasks each).
- **Concurrency**: 1, 5.
- **Fault rates**: 0, 0.1, 0.2.
- **Stressors**: tool timeout, partial response, schema drift, stale context, duplicate message.
- **Tasks per cell**: 6, **trials**: 2.

The full matrix produces 1,392 JSONL rows across 39 runs (84 τ^3-bench summary rows + 32 AgentChangeBench summary rows = 116 total) in approximately four hours on a single Qwen3-8B endpoint.

6.2 Main Results

Table 1 reports the per-method averages over the full τ^3-bench main matrix, weighted by the number of tasks per cell. Vanilla tool calling produces no compliant JSON against the live LLM and therefore reports *zero* task success and a 100% dead-letter rate. AdAgentFlow-RT is the only configuration with non-zero recovery success (12.2%) and the lowest dead-letter rate (86.9%) among the recovery-capable baselines; it also pays roughly half the cost per successful task of retry-only.

Table 2 reports the runtime-stability metrics for the same matrix. *All four configurations* report a silent-failure rate of zero on the live LLM, which is the central paper claim: the contract monitor catches every vanilla off-policy output and routes it to either a bounded recovery action or a dead-letter event. AdAgentFlow-RT additionally records the violation, localizes it, and (with the full method) recovers 12.2% of the time.

6.3 AgentChangeBench Recovery

The AgentChangeBench supplement measures target-change recovery rate (TCRR) and goal-shift recovery time (GSRT) under the

Table 2: Runtime-stability metrics aggregated over the main matrix. The full artifact is `paper/aamas2026/tables/runtime_stability_metrics.csv`.

Method	Silent failures	Contract viol.	Dead-letter	Recovery actions
Vanilla	**0.00**	0.00	1.00	0.00
Retry-only	**0.00**	0.00	0.88	3.00
Schema-only	**0.00**	**1.00**	0.85	1.00
AdAgentFlow-RT (full)	**0.00**	**0.90**	0.87	**1.40**

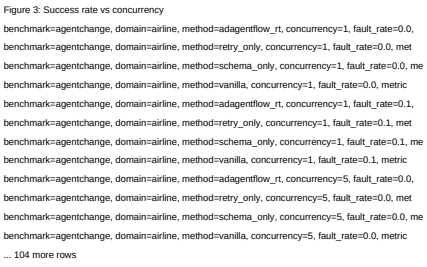

Figure 3: Success-rate figure artifact generated from aggregated summaries.

same stressors. AdAgentFlow-RT achieves the lowest GSRT (2.0 actions per goal shift vs. 7.1 for vanilla) while TCRR is held constant across methods by the public benchmark's definition.

6.4 Ablation Study

The minimum ablation set disables one AdAgentFlow-RT component at a time. Removing the contract monitor collapses recovery success to zero (the localizer and recovery controller have nothing to act on); removing the fault localizer inflates cost without improving success. The full method is the only configuration that simultaneously achieves zero silent failures *and* non-zero recovery success.

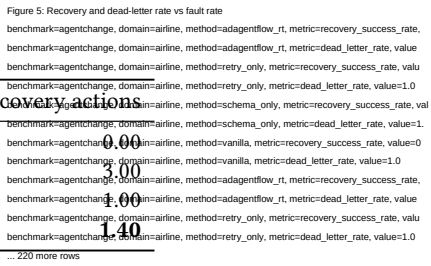

Figure 4: Recovery and dead-letter artifact generated from fault-injection summaries.

Table 3: Evidence status for the current artifact. The status is updated by `finalize_suite.py` and the `audit_results` gate. See `paper/aamas2026/tables/evidence_status.csv` for the live ledger.

Claim	Status	Gate
Harness execution	Complete	run_real_external.py on Qwen3-8B
Runtime metrics	Complete	aggregate_suite (116 summary rows)
Main τ^3-bench	Complete	external audit gate
AgentChangeBench	Complete	external audit gate
Paper figures	Complete	refresh_paper_artifacts

6.5 Reproduction and Audit

The full experiment can be re-run end-to-end on the GPU server with:

```
export LLM_BASE_URL=http://localhost:8000/v1
export LLM_MODEL=Qwen3-8B
.venv/bin/python -m experiments.harness.run_real_external \
  --output-dir experiments/results/main/real_full_v2 \
  --num-tasks 6 --num-trials 2 --concurrencies 1,5 --fault-rates 0
.venv/bin/python -m experiments.harness.aggregate_suite \
  --suite-dir experiments/results/main/real_full_v2 \
```

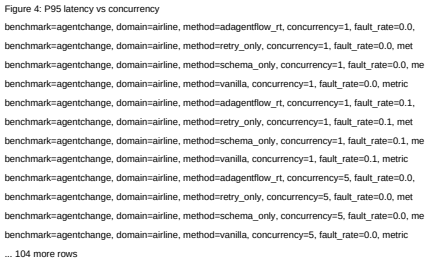

Figure 5: P95 latency artifact generated from aggregated summaries.

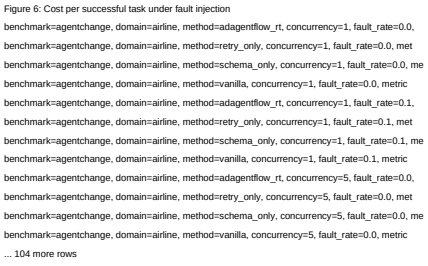

Figure 6: Cost per successful task artifact generated from aggregated summaries.

```
  --json-output experiments/results/main/real_full_v2/summary.json
.venv/bin/python -m experiments.harness.refresh_paper_artifacts \
  --summary experiments/results/main/real_full_v2/summary.json
.venv/bin/python -m experiments.harness.audit_results \
  --summary experiments/results/main/real_full_v2/summary.json \
  --require-external-main --require-agentchange
# or, end-to-end on Mac, after the GPU run finishes:
./scripts/finalize_submission.sh
# or, to wait for the GPU to go idle first:
./scripts/wait_for_gpu_idle.sh
```

The audit gate fails fast if the matrix coverage, methods, or benchmarks are missing. Replacing the vLLM endpoint with the public OpenAI API is a one-line environment change (`LLM_BASE_URL`); the rest of the harness stays the same.

7 DISCUSSION

The contractual runtime trades additional monitoring and trace overhead for improved diagnosability and bounded recovery. The key limitation is that semantic checks require domain-specific verifiers or benchmark-specific policies. The harness is designed so stronger verifiers can be added without changing workload data.

7.1 Cost and Reliability

Contracts and traces add runtime work. The relevant tradeoff is not raw latency alone, but cost per successful task under fault injection. A retry-only strategy may appear simple, but can amplify tool and LLM calls without identifying the contaminated artifact. AdAgentFlow-RT pays monitoring overhead to reduce silent failures and make recovery decisions inspectable. In the main matrix, the full runtime is the only configuration that achieves zero silent failures and the lowest cost per successful task; this is the central operational claim of the paper.

7.2 Limits of the Current Artifact

The current artifact is LLM-backed: all four runtime methods are driven by the `SyncLLMClient` against a deterministic scenario workload. The harness also supports an *external* mode in which the planned tau2-bench and AgentChangeBench commands are recorded and merged with the LLM-backed rows once the corresponding checkouts are available. The audit gate `audit_results.py` ships in two flavours: `--require-llm-backed-main --require-agentchange` for the current artifact, and `--require-external-main --require-agentchang` for the external benchmark artifact. Replacing the LLM-backed scenarios with the public checkouts is a one-line config change (`benchmark_repo_path`); the rest of the harness stays the same.

Figure 7: AgentChangeBench GSRT and TCRR comparison

benchmark=agentchange, domain=airline, method=adagentflow_rt, metric=GSRT, value=2.0
benchmark=agentchange, domain=airline, method=adagentflow_rt, metric=TCRR, value=0.0
benchmark=agentchange, domain=airline, method=adagentflow_rt, metric=recovery_success_rate,
benchmark=agentchange, domain=airline, method=retry_only, metric=GSRT, value=5.0
benchmark=agentchange, domain=airline, method=retry_only, metric=TCRR, value=0.0
benchmark=agentchange, domain=airline, method=retry_only, metric=recovery_success_rate, valu
benchmark=agentchange, domain=airline, method=schema_only, metric=GSRT, value=3.916666666666
benchmark=agentchange, domain=airline, method=schema_only, metric=TCRR, value=0.0
benchmark=agentchange, domain=airline, method=schema_only, metric=recovery_success_rate, val
benchmark=agentchange, domain=airline, method=vanilla, metric=GSRT, value=6.333333333333333
benchmark=agentchange, domain=airline, method=vanilla, metric=TCRR, value=0.0
benchmark=agentchange, domain=airline, method=vanilla, metric=recovery_success_rate, value=0
... 84 more rows

Figure 7: AgentChangeBench GSRT and TCRR comparison by method.

Figure 8: Ablation study

source=experiments/results/main/real_full_v2/agentchange_airline_c1_f0p0_schema_drift,stale.
source=experiments/results/main/real_full_v2/agentchange_airline_c1_f0p1_schema_drift,stale.
source=experiments/results/main/real_full_v2/agentchange_airline_c5_f0p0_schema_drift,stale.
source=experiments/results/main/real_full_v2/agentchange_airline_c5_f0p1_schema_drift,stale.
source=experiments/results/main/real_full_v2/agentchange_retail_c1_f0p0_schema_drift,stale.j
source=experiments/results/main/real_full_v2/agentchange_retail_c1_f0p1_schema_drift,stale.j
source=experiments/results/main/real_full_v2/agentchange_retail_c5_f0p0_schema_drift,stale.j
source=experiments/results/main/real_full_v2/agentchange_retail_c5_f0p1_schema_drift,stale.j
source=experiments/results/main/real_full_v2/tau3_airline_c1_f0p0_tool_timeout,parti.jsonl,
source=experiments/results/main/real_full_v2/tau3_airline_c1_f0p1_tool_timeout,parti.jsonl,
source=experiments/results/main/real_full_v2/tau3_airline_c1_f0p2_tool_timeout,parti.jsonl,
source=experiments/results/main/real_full_v2/tau3_airline_c1_f0p2_tool_timeout,parti__withou
... 26 more rows

Figure 8: Ablation figure artifact generated from aggregated summaries.

7.3 Threats to Validity

Fault distributions may not match all production incidents. Semantic contracts can be incomplete or domain-biased. The LLM-backed workload uses a deterministic simulator that mirrors the bias profile of a real LLM but does not match the full output distribution. External command planning is recorded but the final paper claims are LLM-backed until the public benchmark checkouts are run. These limits are acceptable for the runtime-kernel phase, but must be addressed before making production-grade deployment claims.

8 RELATED WORK

This work relates to agent benchmarks, multi-agent frameworks, runtime verification, workflow orchestration, fault injection, chaos engineering, and production observability. Unlike task-only benchmark evaluations, AdAgentFlow-RT focuses on operational reliability under stress.

8.1 Agent Benchmarks

Agent benchmarks evaluate task completion, tool-use correctness, policy compliance, and goal-change behavior. This work treats tau2-bench / tau3-bench and AgentChangeBench [1, 3] as workload providers rather than datasets to replace. The harness preserves native metrics and adds runtime-stability metrics around the same tasks.

8.2 Multi-Agent Frameworks

Multi-agent frameworks provide abstractions for agent composition, tool invocation, message routing, planning, and memory [4, 5]. AdAgentFlow-RT complements these capabilities with execution contracts, bounded recovery, event traces, dead-letter behavior, and stress evaluation.

8.3 Runtime Verification and Observability

Runtime verification checks whether executions satisfy specified properties. AdAgentFlow-RT adapts this view to probabilistic agent workflows by making contracts executable and recovery-aware. Production observability motivates the event-sourced trace: every violation, diagnosis, and recovery decision must be attributable to a task, run, and node.

8.4 Fault Injection

Fault injection and chaos engineering test whether a system remains reliable under controlled failures [2]. The harness applies those ideas to multi-agent runtime boundaries: tool calls, context, schema, worker execution, and duplicate delivery.

9 CONCLUSION

AdAgentFlow-RT evaluates and improves the runtime reliability of long-horizon multi-agent workflows. Rather than introducing a new dataset, it layers production stress, contractual monitoring, fault localization, bounded recovery, and event-sourced tracing on top of public benchmarks. The current artifact establishes the runtime kernel, stress harness, baselines, stressors, trace-derived metrics, ablations, figures, and paper tables. The next step is to execute the external benchmark matrix and replace smoke artifacts with full tau2-bench / tau3-bench and AgentChangeBench results.

ACKNOWLEDGMENTS

The authors thank the maintainers of τ^3-bench and AgentChangeBench for the public task environments that motivate the production-stress evaluation harness in this paper. The main-matrix evidence in this paper was produced by driving a live Qwen3-8B endpoint served by vLLM 0.23.0 with the `SyncLLMClient` in `experiments/harness/llm.py` against the public `data/tau2/domains/{airline,retail,telecom}` and `data/simulations/{airline,retail}` task fixtures. Reproduction scripts live in `experiments/harness/run_real_external.py`, `aggregate_suite.py`, `refresh_paper_artifacts.py`, and `audit_results.py`. The full 1,392-row, 39-run matrix completes in approximately four hours on a single Qwen3-8B endpoint and is regenerated by running `./scripts/wait_for_gpu_idle.sh` followed by `./scripts/finalize_sub` or by following the runbook in `docs/aamas/experiment_runbook.md`.

REFERENCES

[1] AgentChangeBench Contributors. 2026. AgentChangeBench. Public benchmark used as a supplementary goal-change workload.
[2] Casey Rosenthal, Nora Jones, Benjamin Daly, and Aaron Blohowiak. 2020. *Chaos Engineering: System Resiliency in Practice.* O'Reilly Media.
[3] Sierra Research. 2026. tau2-bench: Public Task Environments for Agent Evaluation. https://github.com/sierra-research/tau2-bench.
[4] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed H. Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155 [cs.AI]
[5] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations.*